Constraint Satisfaction Problems

Chapter 5

Section 1 – 4

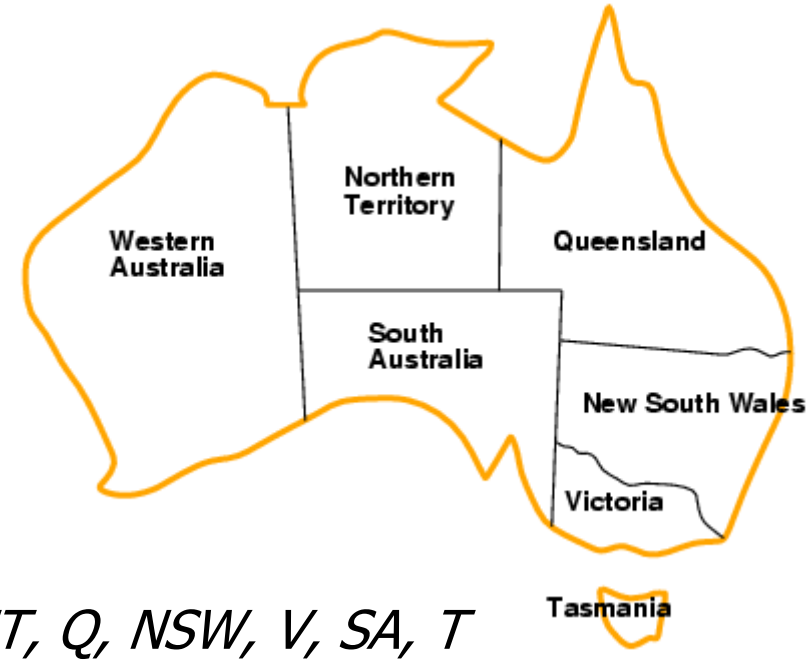
Outline

- Constraint Satisfaction Problems (CSP)
- Backtracking search for CSPs
- Local search for CSPs
- 问题结构

Constraint satisfaction problems (CSPs)

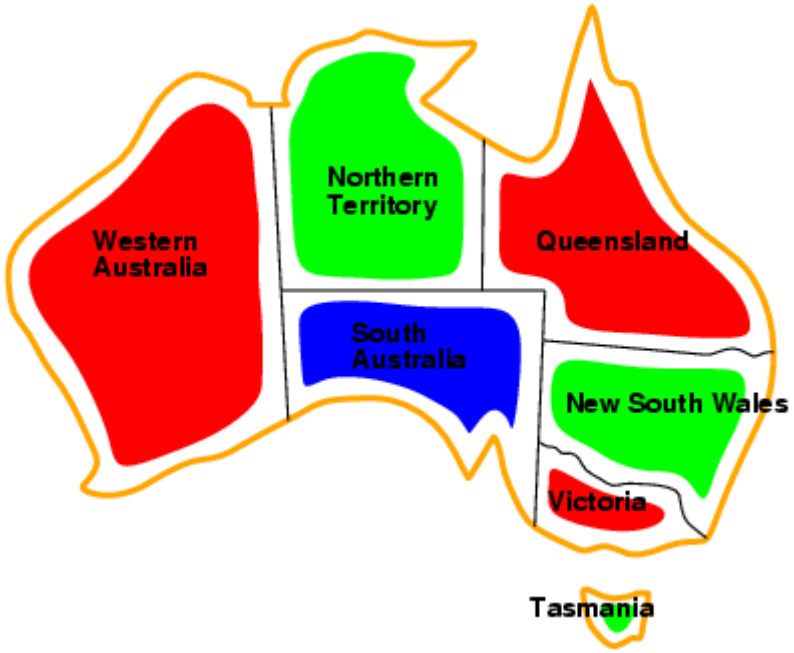
- Standard search problem:
- state is a "black box" – any data structure that supports successor function, heuristic function, and goal test
-
- CSP:
- state is defined by variables X_i with values from domain D_i
-
  - goal test is a set of constraints specifying allowable combinations of values for subsets of variables
  -
- Allows useful general-purpose heuristics
-

Example: Map-Coloring



- Variables *WA, NT, Q, NSW, V, SA, T*
- Domains D_i = {red,green,blue}
- Constraints: adjacent regions must have different colors
-
- e.g., WA ≠ NT, or (WA,NT) in {(red,green),(red,blue),(green,red), (green,blue),(blue,red),(blue,green)}
-

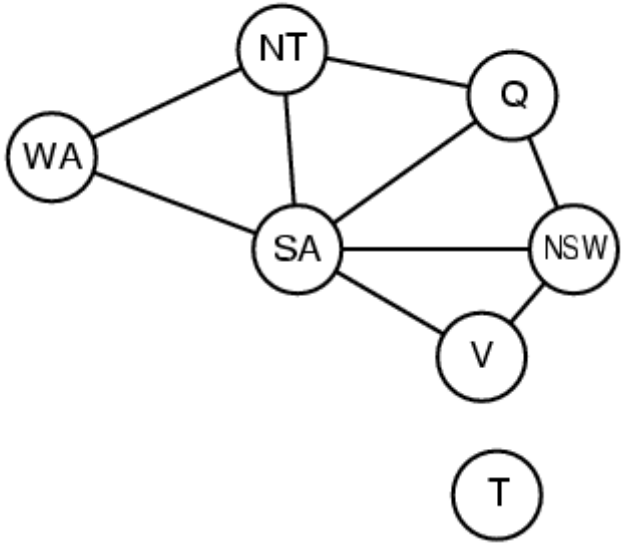
Example: Map-Coloring



- Solutions are complete and consistent assignments, e.g., WA = red, NT = green,Q = red,NSW = green,V = red,SA = blue,T = green
-

Constraint graph

- Binary CSP: each constraint relates two variables
-
- Constraint graph: nodes are variables, arcs are constraints
-



Varieties of CSPs

- Discrete variables
-
  - finite domains:
    - n variables, domain size $d \rightarrow O(d^n)$ complete assignments
    - e.g., Boolean CSPs, incl. Boolean satisfiability (NP-complete)
  - infinite domains:
    - integers, strings, etc.
    - e.g., job scheduling, variables are start/end days for each job
    - need a constraint language, e.g., $StartJob_1 + 5 \leq StartJob_3$
- Continuous variables
-
  - e.g., start/end times for Hubble Space Telescope observations
  - linear constraints solvable in polynomial time by linear programming

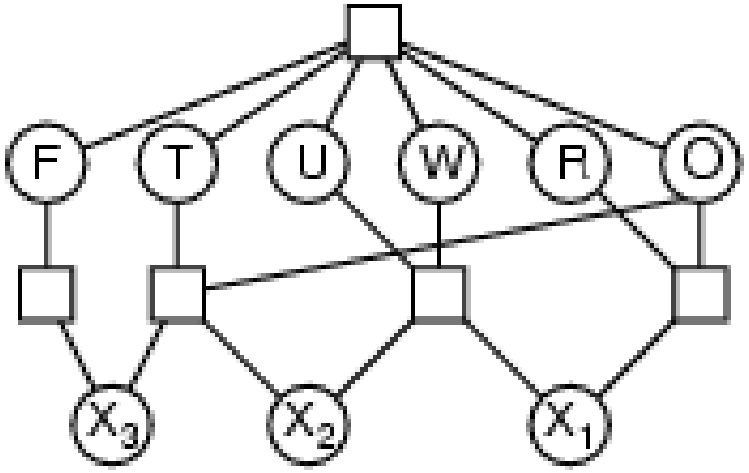
Varieties of constraints

- Unary constraints involve a single variable,
  - e.g., SA ≠ green
  -
- Binary constraints involve pairs of variables,
  - e.g., SA ≠ WA
  -
- Higher-order constraints involve 3 or more variables,
  - e.g., cryptarithmetic column constraints
  -

Preferences (soft constraints), e.g., red is better than $green$
often representable by a cost for each variable assignment
$\longrightarrow$ constrained optimization problems

Example: Cryptarithmetic

```
   T W O
+  T W O
-------
F O U R
```



- Variables: *F T U W R O* X_1 X_2 X_3
- Domains: {*0,1,2,3,4,5,6,7,8,9*}
- Constraints: *Alldiff (F,T,U,W,R,O)*
-
  - $O + O = R + 10 \cdot X_1$
  - $X_1 + W + W = U + 10 \cdot X_2$
  - $X_2 + T + T = O + 10 \cdot X_3$
  - $X_3 = F$, $T \neq 0$, $F \neq 0$
  -

Real-world CSPs

- Assignment problems
  - e.g., who teaches what class
  -
- Timetabling problems
-
  - e.g., which class is offered when and where?
  -
- Transportation scheduling
-
- Factory scheduling
-
- Notice that many real-world problems involve real-valued variables

Standard search formulation (incremental)

States are defined by the values assigned so far

- Initial state: the empty assignment { }
- Successor function: assign a value to an unassigned variable that does not conflict with current assignment
  → fail if no legal assignments

- Goal test: the current assignment is complete

1. This is the same for all CSPs
2. Every solution appears at depth *n* with *n* variables
   → use depth-first search

3.

3.

Backtracking search

- Variable assignments are commutative, i.e.,

[WA = red then NT = green] same as [NT = green then WA = red]

- Only need to consider assignments to a single variable at each node
  - b = d and there are d^n leaves
  -
- Depth-first search for CSPs with single-variable assignments is called backtracking search
-
- Backtracking search is the basic uninformed algorithm for CSPs
-
- Can solve *n*-queens for $n \approx 25$
-

Backtracking search

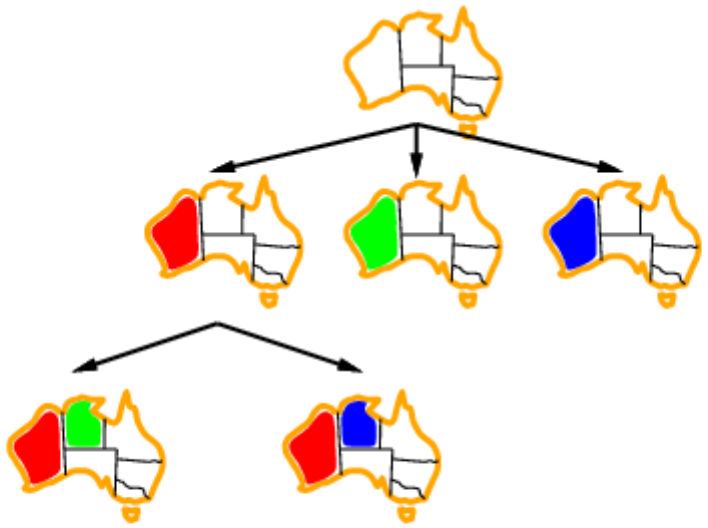
```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
    return RECURSIVE-BACKTRACKING({}, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns a solution, or failure
    if assignment is complete then return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(Variables[csp], assignment, csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment according to Constraints[csp] then
            add { var = value } to assignment
            result ← RECURSIVE-BACKTRACKING(assignment, csp)
            if result ≠ failue then return result
            remove { var = value } from assignment
    return failure
```

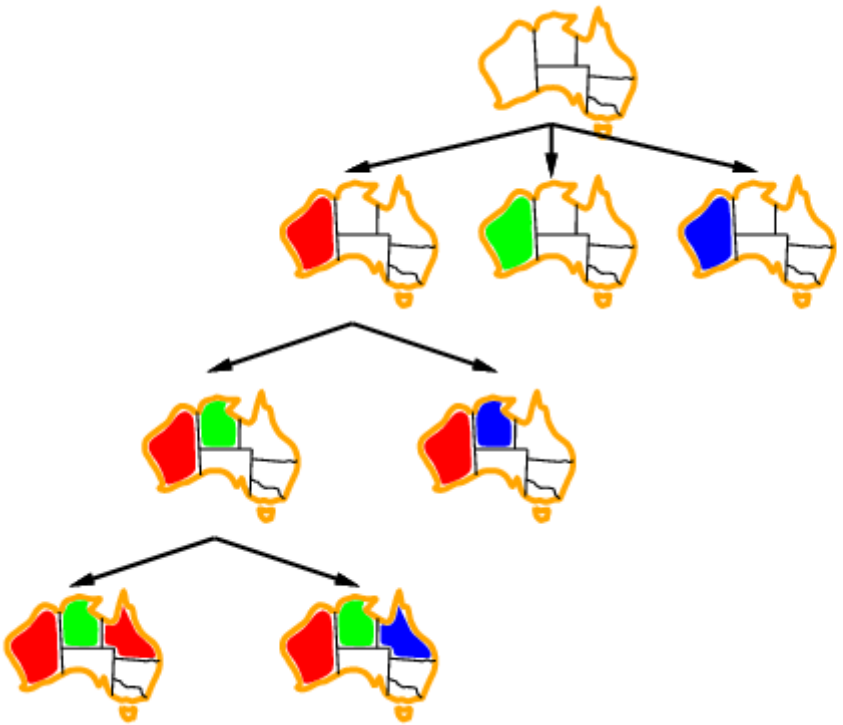
Backtracking example

Backtracking example

Backtracking example



Backtracking example

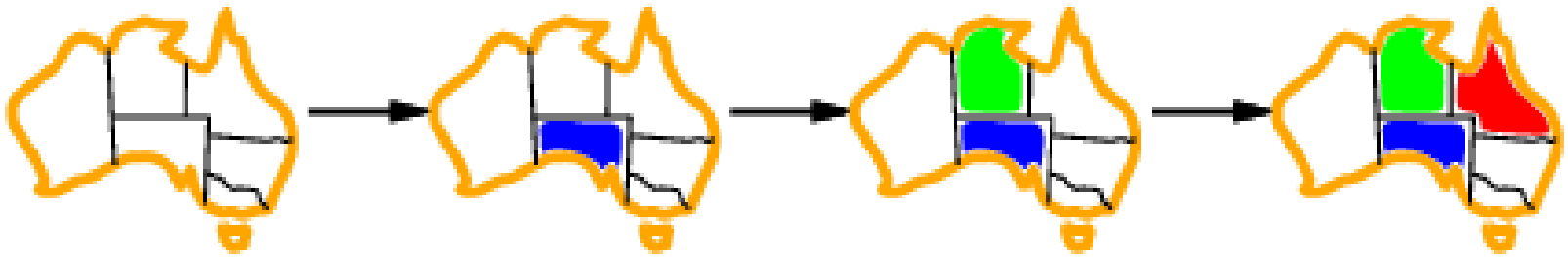


Improving backtracking efficiency

- General-purpose heuristics can give huge gains in speed:
-
  - Which variable should be assigned next?
  -
  - In what order should its values be tried?
  -
  - Can we detect inevitable failure early?
  -

Most constraining variable

- Tie-breaker among most constrained variables
- Most constraining variable:
- (degree heuristic)
  - choose the variable with the most constraints on remaining variables
  -

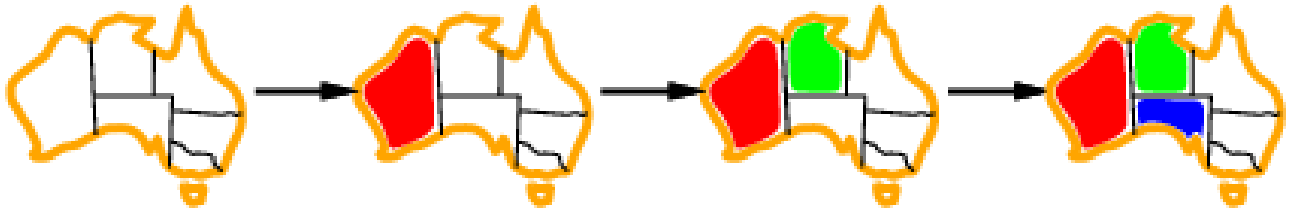


- a.k.a. degree heuristic
-

Most constrained variable

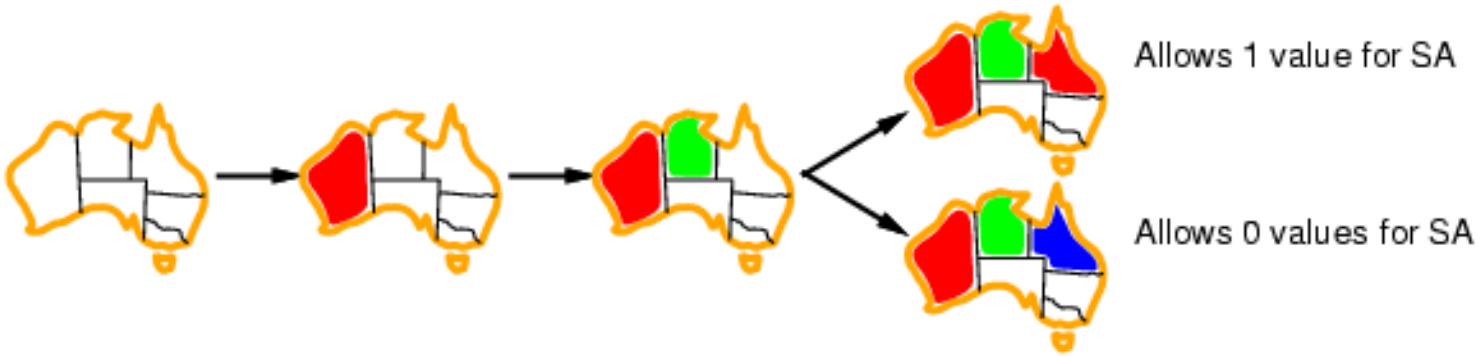
- Most constrained variable:

  choose the variable with the fewest legal values



- a.k.a. minimum remaining values (MRV) heuristic
-

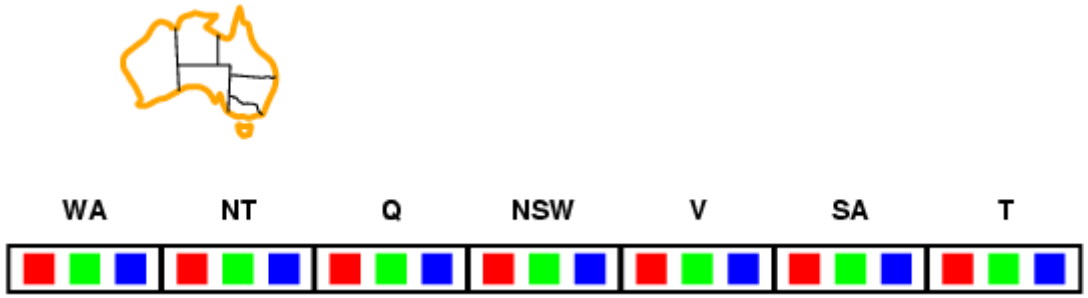
Least constraining value

- Given a variable, choose the least constraining value:
- the one that rules out the fewest values in the remaining variables
- 

- Combining these heuristics makes 1000 queens feasible

Forward checking

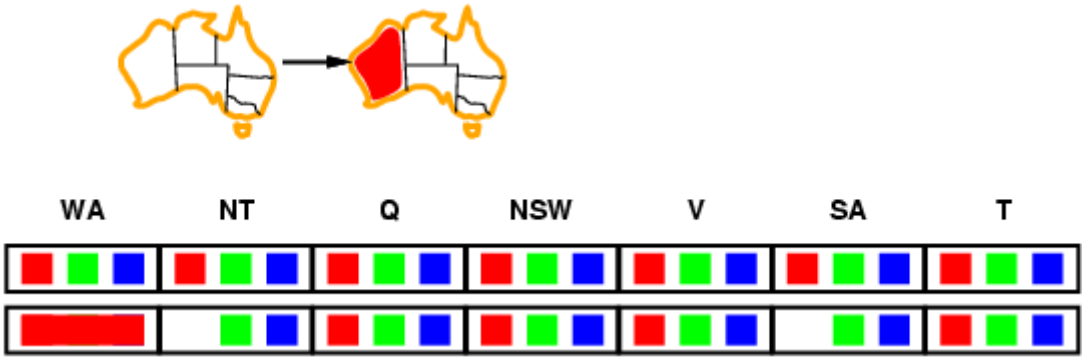
- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values



  - 仅检查已赋值结点的邻居

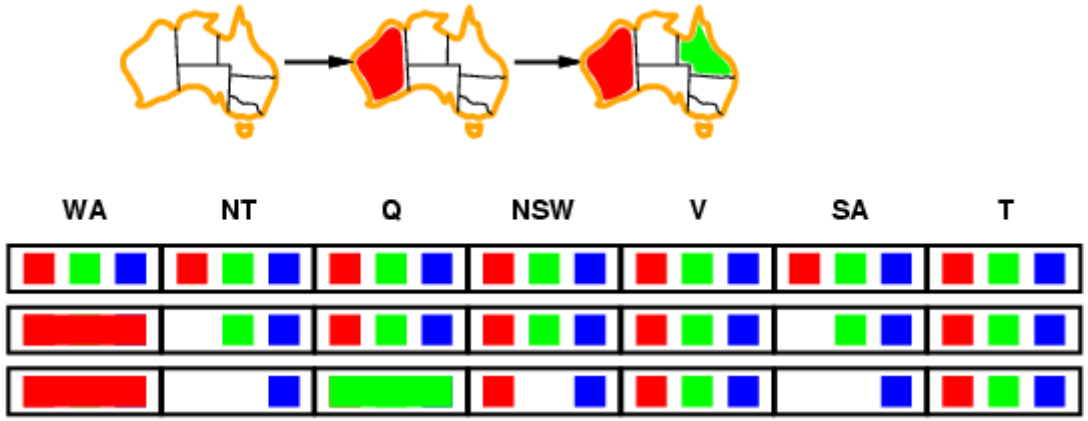
Forward checking

- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values
  -



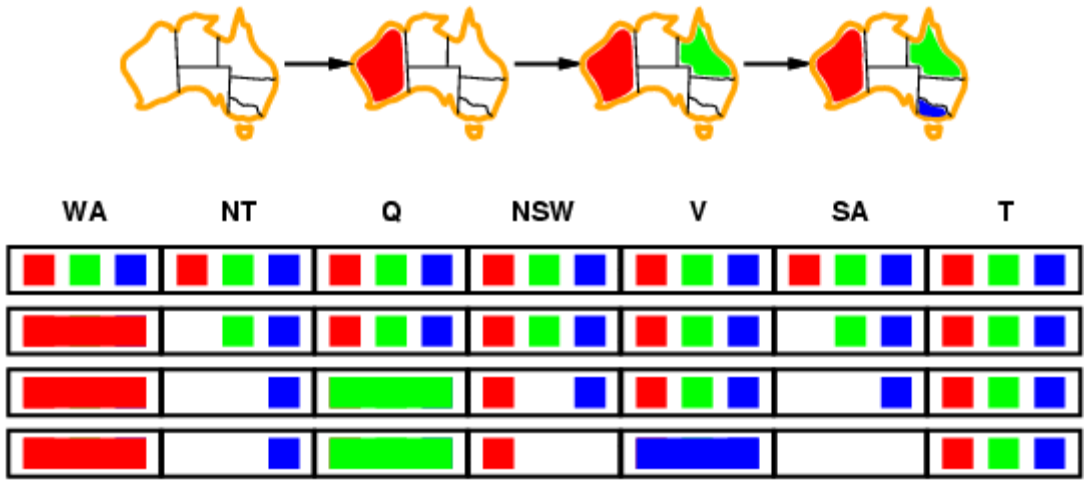
Forward checking

- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values
  -



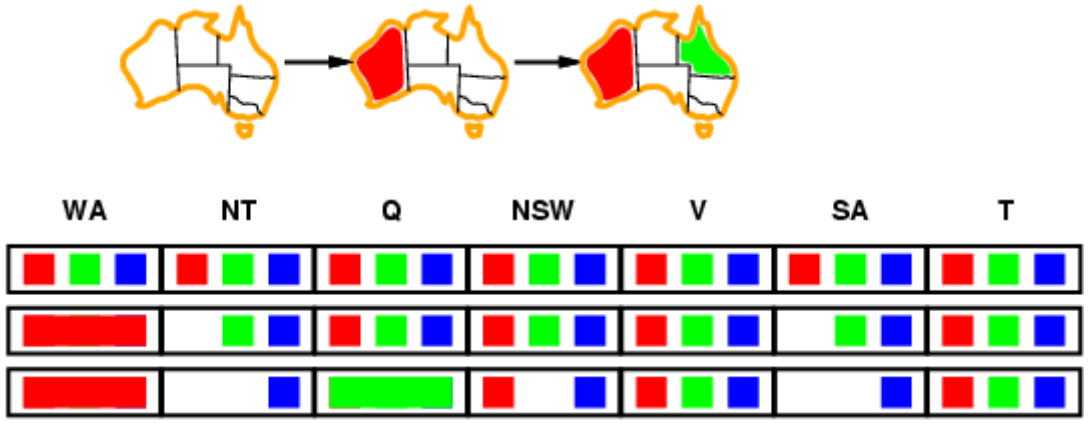
Forward checking

- Idea:
  - Keep track of remaining legal values for unassigned variables
  - Terminate search when any variable has no legal values



  -
  -

Constraint propagation

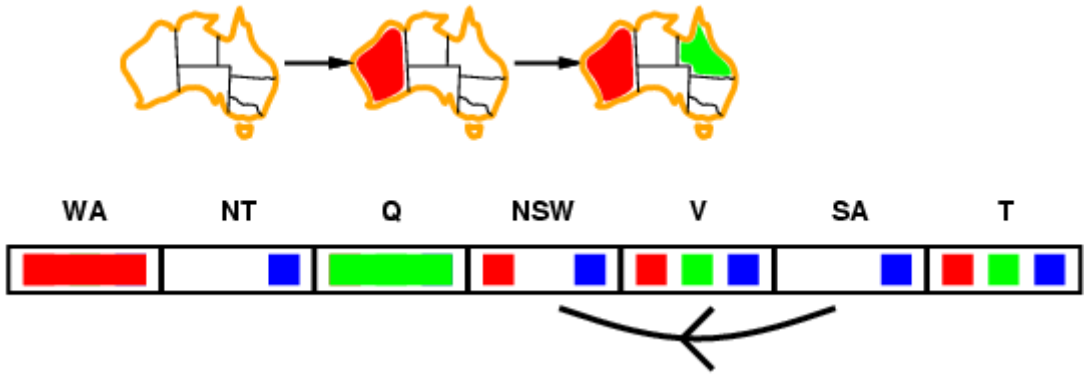
- Forward checking propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures:
- 

- NT and SA cannot both be blue!
- Constraint propagation repeatedly enforces constraints locally
-

Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \rightarrow Y$ is consistent iff
-

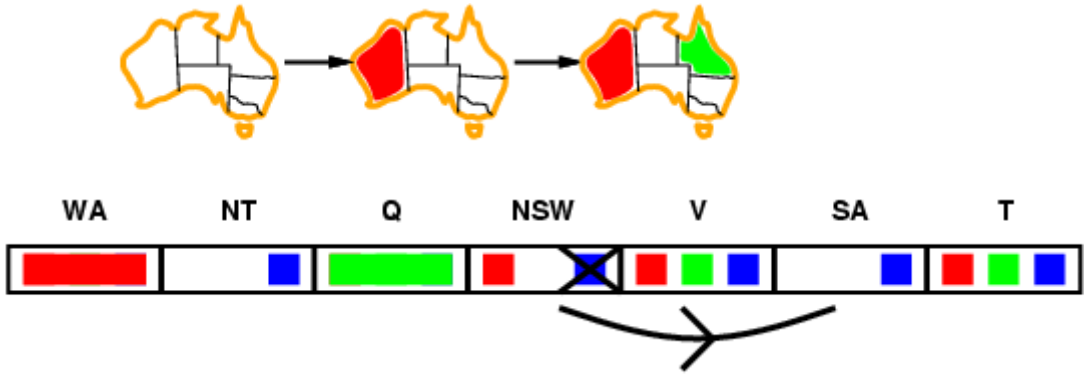
  for every value x of X there is some allowed y



Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \rightarrow Y$ is consistent iff
-

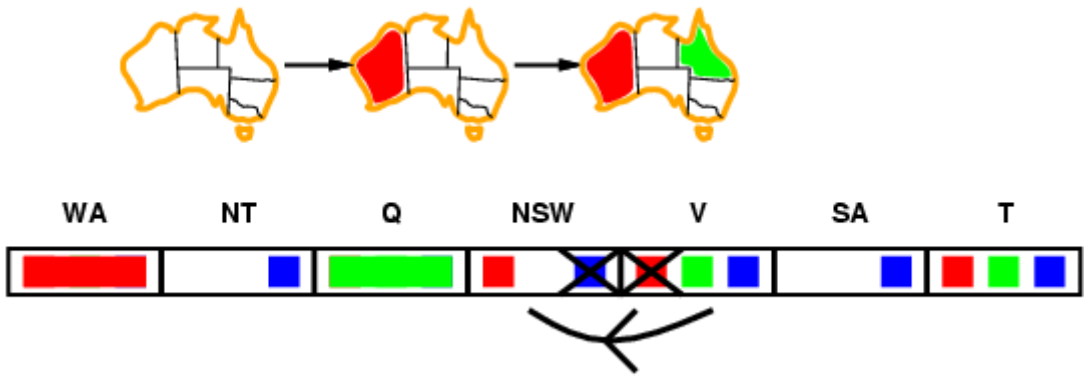
  for every value x of X there is some allowed y



Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \rightarrow Y$ is consistent iff
-

  for every value x of X there is some allowed y

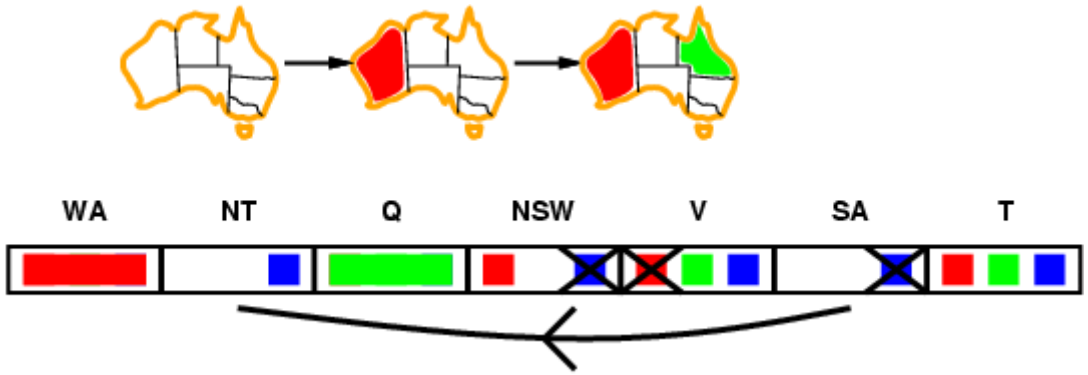


- If X loses a value, neighbors of X need to be rechecked
-

Arc consistency

- Simplest form of propagation makes each arc consistent
- $X \rightarrow Y$ is consistent iff
-

  for every value x of X there is some allowed y



- If X loses a value, neighbors of X need to be rechecked
- Arc consistency detects failure earlier than forward checking
-
- Can be run as a preprocessor or after each assignment
-

Arc consistency algorithm AC-3

```
function AC-3( csp) returns the CSP, possibly with reduced domains
   inputs: csp, a binary CSP with variables {X_1, X_2, ..., X_n}
   local variables: queue, a queue of arcs, initially all the arcs in csp

   while queue is not empty do
      (X_i, X_j) ← REMOVE-FIRST(queue)
      if RM-INCONSISTENT-VALUES(X_i, X_j) then
         for each X_k in NEIGHBORS[X_i] do
            add (X_k, X_i) to queue

---------------------------------------------------------------------------

function RM-INCONSISTENT-VALUES( X_i, X_j) returns true iff remove a value
   removed ← false
   for each x in DOMAIN[X_i] do
      if no value y in DOMAIN[X_j] allows (x,y) to satisfy constraint(X_i, X_j)
         then delete x from DOMAIN[X_i]; removed ← true
   return removed
```

- Time complexity: $O(n^2d^3)$
-

智能回溯

- 对历时回溯的改进
  - 历时回溯：重新访问的是时间最近的决策点
- 智能回溯：回溯到导致失败的变量集合中的一个变量（该集合称为冲突集）
- 例子：
- {Q, NSW, V, T, SA, WA, NT}
- 假设已产生赋值{Q=red, NSW=green, V=blue, T=red} ,给SA赋值导致失败，那么应该回溯到V而不是T。
- 实际上，实现智能回溯很繁琐

Local search for CSPs

- Hill-climbing, simulated annealing typically work with "complete" states, i.e., all variables assigned
- To apply to CSPs:
  - allow states with unsatisfied constraints
  - operators reassign variable values
- Variable selection: randomly select any conflicted variable
- Value selection by min-conflicts heuristic:
- choose value that violates the fewest constraints
- i.e., hill-climb with *h(n)* = total number of violated constraints

MIN-CONFLICTS算法

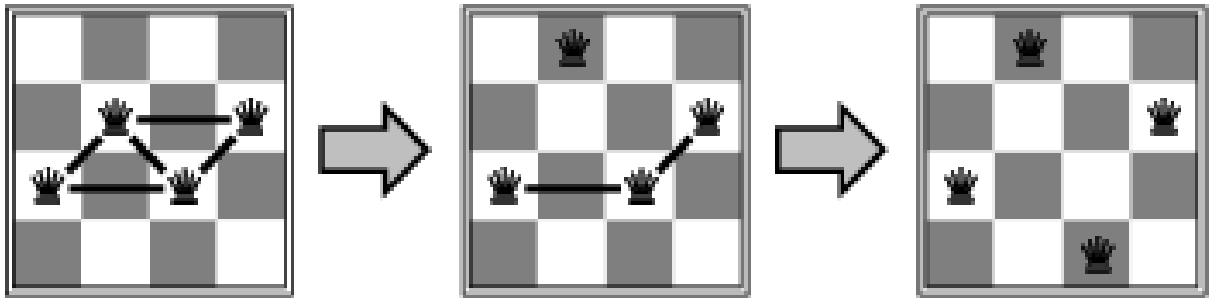
function MIN-CONFLICTS(*csp, max_steps*) **return** solution or failure
inputs: *csp*, a constraint satisfaction problem
max_steps, the number of steps allowed before giving up

current ← an initial complete assignment for *csp*
for *i* = 1 to *max_steps* **do**
if *current* is a solution for *csp* then return *current*
var ← a randomly chosen, conflicted variable from VARIABLES[*csp*]
value ← the value *v* for *var* that minimizes CONFLICTS(*var,v,current,csp*)
set *var* = *value* in *current*
return *faiilure*

与爬山法的差别：爬山法中，CONFLICTS一定减少，此处则未必

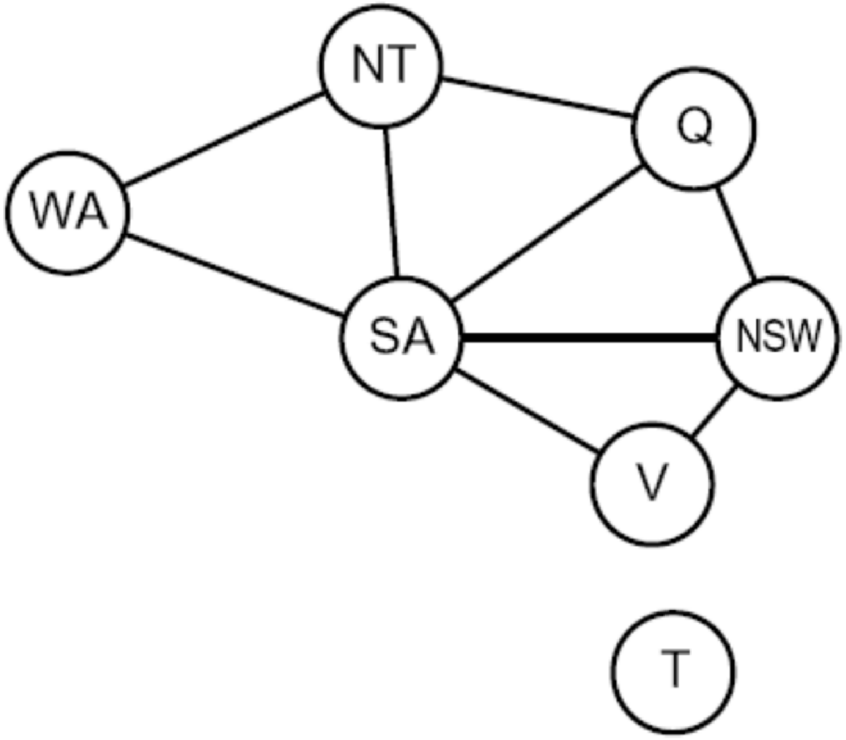
Example: 4-Queens

- States: 4 queens in 4 columns (4^4 = 256 states)
- Actions: move queen in column
- Goal test: no attacks
- Evaluation: *h(n)* = number of attacks
-



- Given random initial state, can solve *n*-queens in almost constant time for arbitrary *n* with high probability (e.g., *n* = 10,000,000)
-

Problem structure



Tasmania and mainland are independent subproblems

Identifiable as connected components of constraint graph

Problem structure contd.

Suppose each subproblem has c variables out of n total

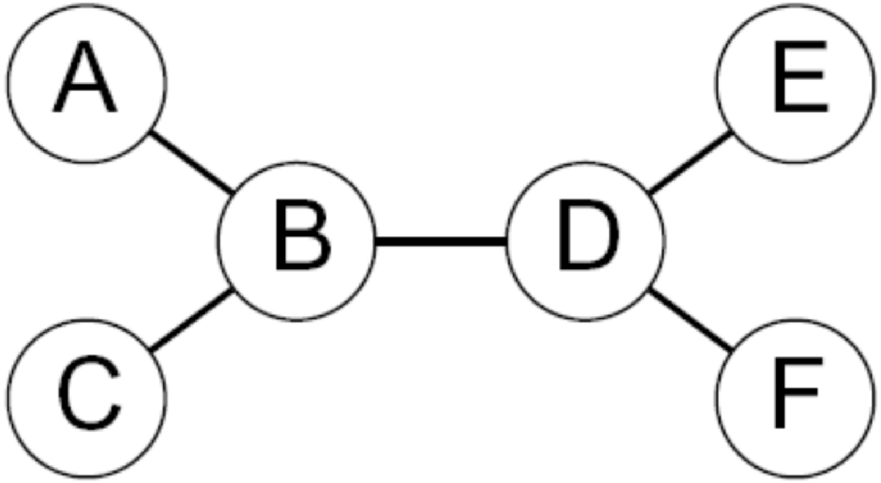
Worst-case solution cost is $n/c \cdot d^c$, **linear** in n

E.g., $n = 80$, $d = 2$, $c = 20$

2^{80} = 4 billion years at 10 million nodes/sec

$4 \cdot 2^{20}$ = 0.4 seconds at 10 million nodes/sec

Tree-structured CSPs



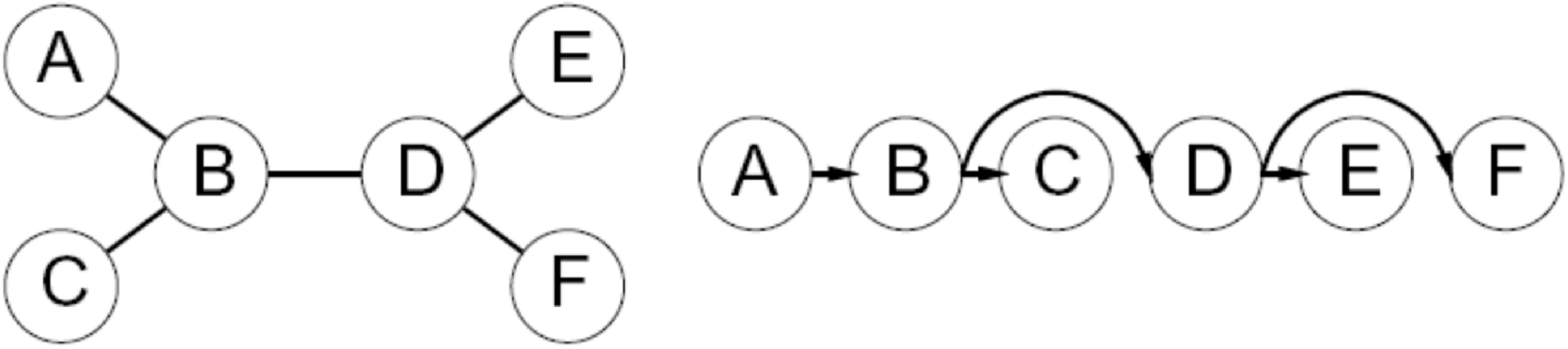
Theorem: if the constraint graph has no loops, the CSP can be solved in $O(n\,d^2)$ time

Compare to general CSPs, where worst-case time is $O(d^n)$

This property also applies to logical and probabilistic reasoning:
an important example of the relation between syntactic restrictions
and the complexity of reasoning.

Algorithm for tree-structured CSPs

1. Choose a variable as root, order variables from root to leaves such that every node's parent precedes it in the ordering

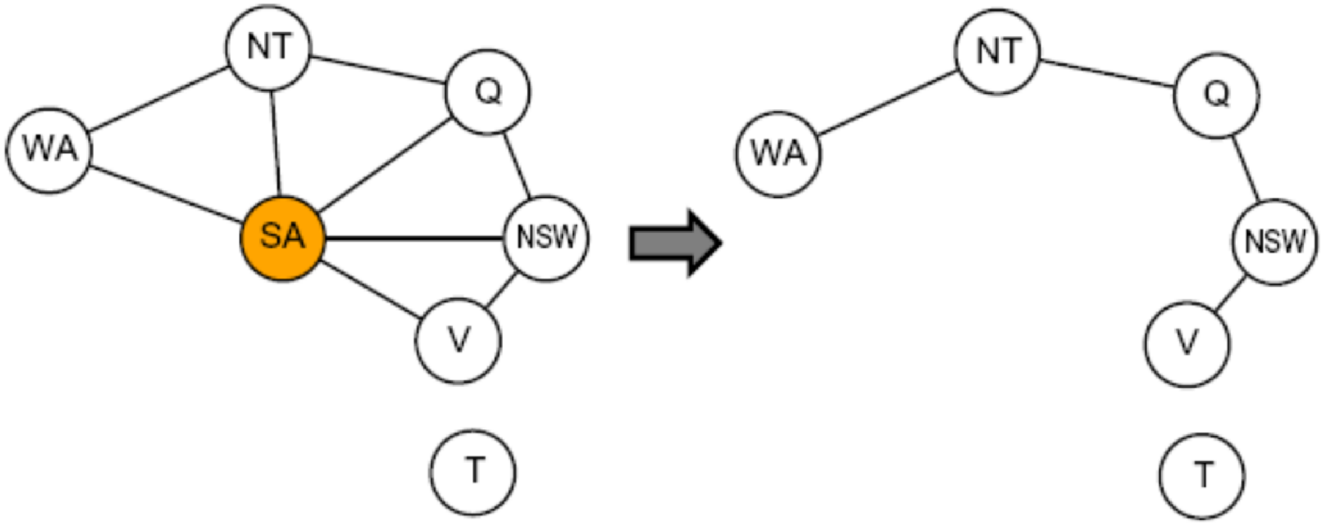


2. For j from n down to 2, apply $\textsc{RemoveInconsistent}(Parent(X_j), X_j)$

3. For j from 1 to n, assign X_j consistently with $Parent(X_j)$

Nearly tree-structured CSPs

Conditioning: instantiate a variable, prune its neighbors' domains



Cutset conditioning: instantiate (in all ways) a set of variables such that the remaining constraint graph is a tree

Cutset size c $\Rightarrow$ runtime $O(d^c \cdot (n-c)d^2)$, very fast for small c

图的树分解

- 把约束图分解为相关联的子问题集
- 独立求解每个子问题
- 合并结果

- Divide-and-Conquer

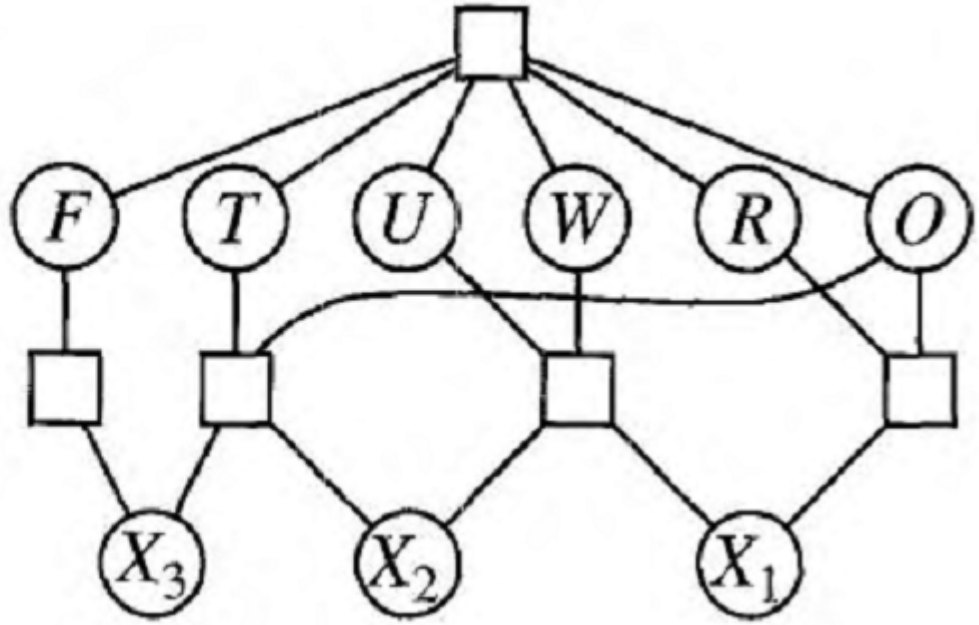
Summary

- CSPs are a special kind of problem:
- states defined by values of a fixed set of variables
-
  - goal test defined by constraints on variable values
  -
- Backtracking = depth-first search with one variable assigned per node
-
- Variable ordering and value selection heuristics help significantly
-
- Forward checking prevents assignments that guarantee later failure
- Constraint propagation (e.g., arc consistency) does additional work to constrain values and detect inconsistencies
-
- Iterative min-conflicts is usually effective in practice
-
- The CSP representation allows analysis of problem structure
- Tree-structured CSPs can be solved in linear time

作业

- 5.6 采用手算、回溯、前向检查、MRV以及

$$\begin{array}{r} T\ W\ O \\ +\ T\ W\ O \\ \hline F\ O\ U\ R \end{array}$$



作业

- Consider the following logic puzzle: In five houses, each with a different color, live 5 persons of different nationalities, each of whom prefer a different brand of cigarette, a different drink, and a different pet. Given the following facts, the question to answer is "Where does the zebra live, and in which house do they drink water?"
  - The Englishman lives in the red house.
  - The Spaniard owns the dog.
  - The Norwegian lives in the first house on the left.
  - Kools are smoked in the yellow house.
  - The man who smokes Chesterfields lives in the house next to the man with the fox.
  - The Norwegian lives next to the blue house.
  - The Winston smoker owns snails.
  - The Lucky Strike smoker drinks orange juice.
  - The Ukrainian drinks tea.
  - The Japanese smokes Parliaments.
  - Kools are smoked in the house next to the house where the horse is kept.
  - Coffee is drunk in the green house.
  - The Green house is immediately to the right (your right) of the ivory house.
  - Milk is drunk in the middle house.
- Discuss different representations of this problem as a CSP. Why would one prefer one representation over another?